OpenC910 配置修改 gem5 RISC-V 配置项

根据 OpenC910 模块与配置汇总，修改 gem5 RISC-V 架构配置文件以匹配 C910 硬件特性。

修改文件列表

1. `src/arch/riscv/RiscvISA.py` - ISA 层配置
2. `src/arch/riscv/RiscvTLB.py` - TLB/MMU 层配置
3. `src/arch/riscv/PMP.py` - 物理内存保护配置
4. `src/arch/riscv/RiscvInterrupts.py` - 中断系统配置
5. `src/arch/riscv/PMAChecker.py` - PMA 检查器
6. `src/arch/riscv/RiscvDecoder.py` - 指令解码器
7. `src/arch/riscv/RiscvFsWorkload.py` - Full System Workload
8. `src/dev/riscv/Plic.py` - 平台级中断控制器
9. `src/dev/riscv/Clint.py` - 核心本地中断控制器

---

1. ISA 层配置 (`src/arch/riscv/RiscvISA.py`)

修改内容

配置项	原值	修改后	C910 依据
`enable_rvv`	`True`	`False`	C910 有 VFPU 但采用 T-Head 扩展，非标准 RVV
`vlen`	`256`	`64`	C910: VEC_WIDTH=63 (非标准，取最接近的 2 的幂)
`elen`	`64`	`64`	C910: FPR_WIDTH=63 (非标准，取最接近的 2 的幂)
`privilege_mode_set`	`MSU`	`MSU`	C910: M/S/U 三特权模式，不支持虚拟化扩展
`enable_Zicbom_fs`	`True`	`False`	C910 配置文件中未见相关定义
`enable_Zicboz_fs`	`True`	`False`	C910 配置文件中未见相关定义
`wfi_resume_on_pending`	`False`	`False`	C910 无此配置，添加注释说明

代码片段

```
class RiscvISA(BaseISA):
    type = "RiscvISA"
    cxx_class = "gem5::RiscvISA::ISA"
    cxx_header = "arch/riscv/isa.hh"

    riscv_type = Param.RiscvType("RV64", "RV32 or RV64")

    # OpenC910: VFPU exists but uses T-Head extensions, not standard RVV
    enable_rvv = Param.Bool(False, "Enable vector extension")
    vlen = Param.RiscvVectorLength(
        64,
```

```
        "Length of each vector register in bits. "
        "VLEN in Ch. 2 of RISC-V vector spec. "
        "OpenC910: VEC_WIDTH=63 (non-standard, closest power of 2 is 64)",
    )
    elen = Param.RiscvVectorElementLength(
        64,
        "Length of each vector element in bits. "
        "ELEN in Ch. 2 of RISC-V vector spec. "
        "OpenC910: FPR_WIDTH=63 (non-standard, closest power of 2 is 64)",
    )
    privilege_mode_set = Param.PrivilegeModeSet(
        "MSU",  # OpenC910: M/S/U three privilege modes, no hypervisor
        "The combination of privilege modes "
        "in Privilege Levels section of RISC-V privileged spec. "
        "OpenC910: MSU (Machine/Supervisor/User) without hypervisor",
    )

    # OpenC910: No Zicbom/Zicboz support detected
    enable_Zicbom_fs = Param.Bool(False, "Enable Zicbom extension in FS
mode")
    enable_Zicboz_fs = Param.Bool(False, "Enable Zicboz extension in FS
mode")
    enable_Zcd = Param.Bool(
        True,
        "Enable Zcd extensions. "
        "Set the option to false implies the Zcmp and Zcmt is enable as "
        "c.fsdsp is overlap with them."
        "Refs: https://github.com/riscv/riscv-isa-
manual/blob/main/src/zc.adoc",
    )
    enable_Smrnmi = Param.Bool(
        False, "Resumable non-maskable interrupt in FS mode"
    )

    wfi_resume_on_pending = Param.Bool(
        False,
        "If wfi_resume_on_pending is set to True, the hart will resume "
        "execution when interrupt becomes pending. The local enabled status
"
        "is not considered.\n"
        "If wfi_resume_on_pending is set to False, the hart will only "
        "resume the execution when an locally enabled interrupt becomes "
        "pending.\n"
        "OpenC910: This feature is not implemented",
    )
```

---

2. TLB/MMU 层配置 (`src/arch/riscv/RiscvTLB.py`)

修改内容

配置项	原值	修改后	C910 依据

配置项	原值	修改后	C910 依据
size	64	1024	C910: JTLB_ENTRY_1024 (可选 2048)
num_squash_per_cycle	4	4	添加注释：gem5 仿真参数，RTL 无对应

代码片段

```
class RiscvPagetableWalker(ClockedObject):
    type = "RiscvPagetableWalker"
    cxx_class = "gem5::RiscvISA::Walker"
    cxx_header = "arch/riscv/pagetable_walker.hh"

    port = RequestPort("Port for the hardware table walker")
    system = Param.System(Parent.any, "system object")
    num_squash_per_cycle = Param.Unsigned(
        4, "Number of outstanding walks that can be squashed per cycle. "
        "OpenC910: Not applicable (gem5 仿真参数，RTL 无对应)"
    )
    pma_checker = Param.BasePMAChecker(Parent.any, "PMA Checker")
    pmp = Param.PMP(Parent.any, "PMP")


class RiscvTLB(BaseTLB):
    type = "RiscvTLB"
    cxx_class = "gem5::RiscvISA::TLB"
    cxx_header = "arch/riscv/tlb.hh"

    size = Param.Int(1024, "TLB size. OpenC910: JTLB_ENTRY_1024 (可选
2048)")
    walker = Param.RiscvPagetableWalker(
        RiscvPagetableWalker(), "page table walker"
    )
    pma_checker = Param.BasePMAChecker(Parent.any, "PMA Checker")
    pmp = Param.PMP(Parent.any, "Physical Memory Protection Unit")
```

---

3. 物理内存保护配置 (src/arch/riscv/PMP.py)

修改内容

配置项	原值	修改后	C910 依据
pmp_entries	16	8	C910: PMP_REGION_8 (固定 8 项)

代码片段

```
class PMP(SimObject):
    type = "PMP"
    cxx_header = "arch/riscv/pmp.hh"
```

```
    cxx_class = "gem5::RiscvISA::PMP"

    pmp_entries = Param.Int(8, "Maximum PMP Entries Supported. OpenC910:
PMP_REGION_8 (固定 8 项)")
```

---

4. 中断系统配置 (src/arch/riscv/RiscvInterrupts.py)

修改内容

配置项	原注释	修改后	C910 依据
local_interrupt_pins	"Pins for local interrupts"	添加 "OpenC910: CLINT + PLIC"	C910 中断架构
local_interrupt_ids	"list of local interrupt ids"	添加 "OpenC910: PLIC_INT_NUM=144"	C910: 144 个中断源
nmi_cause	"Non-maskable interrupt(NMI) cause"	添加 "OpenC910: Not implemented"	C910 无可编程 NMI cause

代码片段

```
class RiscvInterrupts(BaseInterrupts):
    type = "RiscvInterrupts"
    cxx_class = "gem5::RiscvISA::Interrupts"
    cxx_header = "arch/riscv/interrupts.hh"

    local_interrupt_pins = VectorIntSinkPin(
        "Pins for local interrupts. OpenC910: CLINT + PLIC"
    )
    local_interrupt_ids = VectorParam.Unsigned(
        [], "list of local interrupt ids. OpenC910: PLIC_INT_NUM=144 (144
个中断源)"
    )
    nmi_cause = Param.Int(
        0, "Non-maskable interrupt(NMI) cause. OpenC910: Not implemented"
    )
```

---

5. PMA 检查器配置 (src/arch/riscv/PMAChecker.py)

修改内容

配置项	原注释	修改后	C910 依据
misaligned	Address ranges support...	添加 "OpenC910: Hardware supports misaligned access"	C910 硬件支持非对齐访问

配置项	原注释	修改后	C910 依据
`uncacheable`	Uncacheable address ranges	添加 "OpenC910: PMA checker implemented implicitly via BIU/CIU"	C910 通过 BIU/CIU 隐式实现

代码片段

```
class PMAChecker(BasePMAChecker):
    type = "PMAChecker"
    cxx_header = "arch/riscv/pma_checker.hh"
    cxx_class = "gem5::RiscvISA::PMAChecker"

    # OpenC910: PMA checker implemented implicitly via BIU/CIU
    uncacheable = VectorParam.AddrRange([], "Uncacheable address ranges")
    misaligned = VectorParam.AddrRange(
        [],
        "Address ranges support misaligned load/store to memory. "
        "OpenC910: Hardware supports misaligned access",
    )
```

---

6. 指令解码器配置 (`src/arch/riscv/RiscvDecoder.py`)

修改内容

配置项	原注释	修改后	C910 依据
class doc	无	添加 "OpenC910: ct_ifu_ipdecode.v and other IFU decoder units"	C910 有完整取指/译码单元

代码片段

```
class RiscvDecoder(InstDecoder):
    type = "RiscvDecoder"
    cxx_class = "gem5::RiscvISA::Decoder"
    cxx_header = "arch/riscv/decoder.hh"
    # OpenC910: ct_ifu_ipdecode.v and other IFU decoder units
```

---

7. Full System Workload 配置 (`src/arch/riscv/RiscvFsWorkload.py`)

修改内容

配置项	原注释	修改后	C910 依据

配置项	原注释	修改后	C910 依据
`semihosting`	Enable support...	添加 "OpenC910: Not applicable (gem5 仿真器特性，RTL 无需实现)"	C910 是硬件实现，无需 semihosting

代码片段

```
class RiscvLinux(KernelWorkload):
    type = "RiscvLinux"
    cxx_class = "gem5::RiscvISA::FsLinux"
    cxx_header = "arch/riscv/linux/fs_workload.hh"

    # OpenC910: These are gem5 simulator-specific parameters,
    # not part of the C910 RTL implementation
    dtb_filename = Param.String(
        "", "File that contains the Device Tree Blob. Don't use DTB if
empty."
    )
    dtb_addr = Param.Addr(0x87E00000, "DTB address")
    semihosting = Param.RiscvSemihosting(
        NULL,
        "Enable support for RISC-V semihosting by settings this parameter.
"
        "OpenC910: Not applicable (gem5 仿真器特性，RTL 无需实现)",
    )
```

---

8. 平台级中断控制器 (`src/dev/riscv/Plic.py`)

修改内容

配置项	原值	修改后	C910 依据
`n_src`	无默认值	`144`	C910: PLIC_INT_NUM=144 (144 个中断源)
`hart_config`	无 C910 说明	添加 "OpenC910: PLIC_HART_NUM, PLIC_ID_NUM=10, PLIC_PRIO_BIT=5"	C910: 10 个中断 ID, 5 位优先级

代码片段

```
class Plic(PlicBase):
    type = "Plic"
    cxx_header = "dev/riscv/plic.hh"
    cxx_class = "gem5::Plic"
    pio_size = 0x4000000
    n_src = Param.Int(
        144,
```

```
        "Number of interrupt sources. "
        "OpenC910: PLIC_INT_NUM=144 (144 个中断源)",
    )
    hart_config = Param.String(
        "",
        "String represent for PLIC hart/pmode config like QEMU plic. "
        "OpenC910: PLIC_HART_NUM = actual Hart count, PLIC_ID_NUM=10,
PLIC_PRIO_BIT=5. "
        "Ex."
        "'M'              1 hart with M mode"
        "'MS,MS'          2 harts, 0-1 with M and S mode"
        "'M,MS,MS,MS,MS'  5 harts, 0 with M mode, 1-5 with M and S mode",
    )
```

---

9. 核心本地中断控制器 (src/dev/riscv/Clint.py)

修改内容

配置项	原值	修改后	C910 依据
num_threads	无默认值	1	C910: MAX_HART_NUM=32 (最多 32 个 Hart)

代码片段

```
class Clint(BasicPioDevice):
    type = "Clint"
    cxx_header = "dev/riscv/clint.hh"
    cxx_class = "gem5::Clint"
    int_pin = IntSinkPin("Pin to receive RTC signal")
    pio_size = Param.Addr(0xC000, "PIO Size")
    num_threads = Param.Int(
        1,
        "Number of threads in the system. "
        "OpenC910: MAX_HART_NUM=32 (最多 32 个 Hart), PLIC_HART_NUM = actual
count",
    )
```

为什么只修改了这 9 个模块?

根据 PDF 文档的对比分析，C910 中的许多模块在 gem5 中**没有直接对应的配置项**，原因如下：

C910 有但 gem5 无对应配置的模块

C910 模块	gem5 对应	说明
BIU (总线接口单元)	无直接对应	C910 AXI 总线接口，gem5 通过 System/MemBus 抽象

C910 模块	gem5 对应	说明
CIU (缓存接口单元)	无直接对应	C910 缓存一致性接口，gem5 通过 Cache/MemorySystem 处理
IFU (指令取指单元)	简化模型	C910 有完整取指/预取逻辑 (icache, btb, bht, ras, lbuf)
IDU (指令发射单元)	简化模型	C910 有多发射队列 (AIQ/BIQ/LSIQ/SDIQ/VIQ)
IU (整数单元)	简化模型	C910 有完整 ALU/乘除单元
LSU (加载存储单元)	简化模型	C910 有完整 load/store 流水线 (dcache, lq, sq, lfb, wmb, pfu)
RTU (退休单元)	简化模型	C910 有完整 ROB
L2C (L2 缓存)	可配置	C910 集成 16 路组相联 L2 (128K-8M 可选)
HAD (硬件调试)	GDB 远程调试	C910 有 ETM/断点硬件模块
PMU (性能计数器)	无直接对应	C910: HPCP 16 计数器
VFPU (向量浮点)	enable_rvv	C910 是 T-Head 扩展，非标准 RVV
CP0 (协处理器)	无对应	C910 协处理器接口
CLINT/PLIC	简化模型	C910 完整实现
SAB (Snoop 地址缓冲)	无对应	C910: SAB_DEPTH=24, SAB_RDEPTH=16, SAB_WDEPTH=8

gem5 有但 C910 未实现的配置项

1. `enable_Smrnmi` - 可恢复 NMI (gem5 特有)
2. `wfi_resume_on_pending` - WFI 中断恢复 (gem5 特有)
3. `nmi_cause` - 可编程 NMI cause (gem5 特有)
4. `num_squash_per_cycle` - gem5 仿真参数
5. **所有 Semihosting/Workload 相关** - 仿真器特性
6. **CPU 模型 (AtomicSimpleCPU/MinorCPU/O3CPU)** - gem5 仿真模型

修改的 9 个模块是**有参数可配置**的模块

这 9 个模块是 gem5 中**存在可配置参数**且**与 C910 硬件特性对应**的模块：

1. **RiscvISA.py** - ISA 扩展配置 (RV64, 向量扩展，特权模式)
2. **RiscvTLB.py** - TLB 大小配置
3. **PMP.py** - PMP 条目数配置
4. **RiscvInterrupts.py** - 中断配置
5. **PMAChecker.py** - PMA 检查器 (地址范围配置)
6. **RiscvDecoder.py** - 指令解码器 (仅添加注释)
7. **RiscvFsWorkload.py** - Workload 配置 (添加注释说明)
8. **Plic.py** - PLIC 中断源数量
9. **Clint.py** - CLINT 线程数

其他 C910 模块 (如 IFU, IDU, LSU, RTU, L2C 等) 在 gem5 中是**CPU 微架构模型**的一部分，不是通过 Python 配置文件直接设置参数，而是通过选择不同的 CPU 模型 (AtomicSimpleCPU, MinorCPU, O3CPU) 来间接影响。

C910 不存在/未实现的 gem5 配置项

以下配置项在 C910 中**不存在**或**未实现**，保持禁用状态：

1. `enable_Smrnmi` - 可恢复 NMI (Smrnmi)
2. `wfi_resume_on_pending` - WFI 中断恢复
3. `nmi_cause` - 可编程 NMI cause
4. `enable_Zicbom_fs` - 缓存块管理指令
5. `enable_Zicboz_fs` - 缓存块清零指令
6. `num_squash_per_cycle` - gem5 仿真参数
7. **所有 Semihosting/Workload 相关** - 仿真器特性，RTL 无需实现

C910 与 gem5 的主要差异

特性	gem5	OpenC910
模型类型	软件仿真模型，配置灵活	硬件 RTL 实现，参数在综合时固定
向量扩展	标准 RVV	T-Head VFPU 扩展 (非标准)
多核一致性	可配置	完整的 BIU/CIU/SAB 硬件模块
调试支持	GDB 远程调试	完整 HAD/ETM/断点硬件模块
性能监控	基础支持	完整 PMU/HPCP (16 计数器)
中断控制器	简化模型	完整 PLIC (144 源) + CLINT
L2 缓存	可配置	集成 16 路组相联 L2 (128K-8M 可选)

参考文档

- OpenC910 模块与配置汇总 (步骤二.pdf)
- RISC-V 特权级规范 (Privileged Specification)
- RISC-V 向量扩展规范 (Vector Extension)